CURS:	DAM2	**DATA:**	Octubre-2026	**Qualificació**	
MÒDUL:	M0490 - Programació Serveis i Processos				
TEMA:	Criptografia - Integritat				
RA	RA5				
Professors	Cristina Reina - Carles Bonet				

Enunciat:

Les funcions hash són una eina fonamental en seguretat informàtica. Permeten generar un resum de mida fixa a partir de dades d'entrada i s'utilitzen, entre altres aplicacions, per comprovar la integritat de la informació i protegir les contrasenyes emmagatzemades en sistemes informàtics.

En aquesta pràctica desenvoluparàs una aplicació en Java que permetrà experimentar amb les funcions hash, verificar la integritat dels missatges i implementar un sistema bàsic de registre i autenticació d'usuaris mitjançant hashes i salts (*salt*).

Part 1: Càlcul de resums

Desenvolupa un programa que permeti introduir missatges i calcular-ne el hash mitjançant l'algorisme SHA-256.

El programa ha de permetre:

1. Introduir un missatge per teclat i mostrar-ne el hash.

```
Entra text: Hola Carles
Hash: C524C18E13262416F0A9933989ABD71647B0780A22E413E9EEA250A26BD4EE393A6CD47A3D96C2E14518EC650777DF527AA60EBA0A3E05C188E00556B0ECB211
PS D:\DAM2\ServeisProcessos\Encriptacio\PE02HashSalt>
```

2. Introduir dos missatges i comprovar si els seus hashes coincideixen.

```
Entra text: Hola Carles
Hash: C524C18E13262416F0A9933989ABD71647B0780A22E413E9EEA250A26BD4EE393A6CD47A3D96C2E14518EC650777DF527AA60EBA0A3E05C188E00556B0ECB211
```

```
Entra text: Hola Carles
Hash: C524C18E13262416F0A9933989ABD71647B0780A22E413E9EEA250A26BD4EE393A6CD47A3D96C2E14518EC650777DF527AA60EBA0A3E05C188E00556B0ECB211
```

3. Calcular el hash d'un missatge, modificar-ne el contingut i tornar-lo a calcular per observar-ne les diferències.

```
Entra text: Hola Carles
Hash: C524C18E13262416F0A9933989ABD71647B0780A22E413E9EEA250A26BD4EE393A6CD47A3D96C2E14518EC650777DF527AA60EBA0A3E05C188E00556B0ECB211
```

```
Entra text: Hola Carless
Hash: D41DE69734D562469415487E4267EC8C00E5B07CE5CC23851DBBC64848B774AC61AA32FC5A4268FDBDCA2B84BB0A8B0A95662CAF5EA71B0FE5EFBFE66D431261
```

4. **Calcular diverses vegades el hash d'un mateix missatge i comprovar si el resultat es manté.**

```
Entra text: hola
Hash: E83E8535D6F689493E5819BD60AA3E5FDCBA940E6D111AB6FB5C34F24F86496BF3726E2BF4EC59D6D2F5A2AEB1E4F103283E7D64E4F49C03B4C4725CB361E773
Entra text: hola
Hash: E83E8535D6F689493E5819BD60AA3E5FDCBA940E6D111AB6FB5C34F24F86496BF3726E2BF4EC59D6D2F5A2AEB1E4F103283E7D64E4F49C03B4C4725CB361E773
Entra text: hola
Hash: E83E8535D6F689493E5819BD60AA3E5FDCBA940E6D111AB6FB5C34F24F86496BF3726E2BF4EC59D6D2F5A2AEB1E4F103283E7D64E4F49C03B4C4725CB361E773
Entra text: hola
Hash: E83E8535D6F689493E5819BD60AA3E5FDCBA940E6D111AB6FB5C34F24F86496BF3726E2BF4EC59D6D2F5A2AEB1E4F103283E7D64E4F49C03B4C4725CB361E773
```

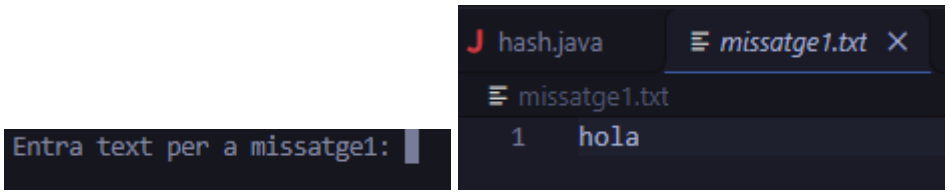
Els hashes s'han de mostrar en format hexadecimal.

Part 2: Verificació de la integritat dels missatges

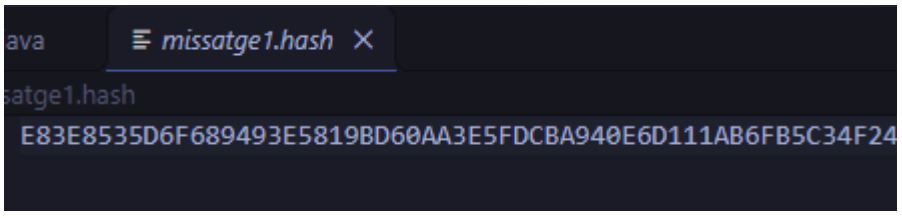
Implementa una funcionalitat que simuli l'enviament d'un missatge entre dos usuaris.

El programa haurà de:

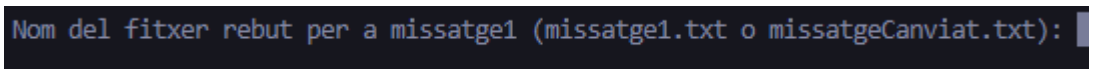
1. **Demanar a l'emissor el contingut d'un missatge i el desarà en un fitxer "missatge.txt".**



2. **Calcular i guardar temporalment el hash original en un fitxer "missatge.hash".**



3. **Simular la recepció del missatge.**
4. **Permetre introduir el missatge rebut, que podrà ser igual o diferent de l'original: Introduir el nom del fitxer a llegir des de teclat. Pot ser el fitxer original "missatge.txt" o el "missatgeCanviat.txt" (que conté una versió modificada de l'original.**



5. **Calcular el hash del missatge rebut (fitxer indicat) i comparar-lo amb el hash original (missatge.hash).**

```
Nom del fitxer rebut per a missatge1 (missatge1.txt o missatgeCanviat.txt): missatge1.txt
Buscant: D:\DAM2\ServeisProcessos\Encriptacio\PE02HashSalt\missatge1.txt
Els hashes coincideixen: no s'ha modificat.
```

6. **Indicar si els dos hashes coincideixen o si s'ha detectat una modificació.**

```
Entra text per a missatge2: hola
Hash: E83E8535D6F689493E5819BD60AA3E5FDCBA940E6D111AB6FB5C34F24F86496BF3726E2BF4EC59D6D2F5A2AEB1E
3B4C4725CB361E773
Nom del fitxer rebut per a missatge2 (missatge2.txt o missatgeCanviat.txt): missatgeCanviat.txt
Buscant: D:\DAM2\ServeisProcessos\Encriptacio\PE02HashSalt\missatgeCanviat.txt
S'ha detectat una modificació.
```

Aquesta part ha de permetre comprovar experimentalment com una modificació del missatge afecta el seu hash.

Part 3: Registre d'usuaris amb salt

Desenvolupa un sistema de registre que permeti crear usuaris i emmagatzemar les seves credencials de manera que les contrasenyes originals no es guardin en text pla.

Per a cada usuari, el programa haurà de:

1. **Demanar un nom d'usuari i una contrasenya.**

```
Nom d'usuari (buit per acabar):
```

2. **Generar una salt aleatòria i diferent per a cada registre.**
3. **Calcular el hash de la contrasenya utilitzant la salt generada.**
4. **Emmagatzemar el nom d'usuari, la salt i el hash resultant.**

```
--- Usuaris emmagatzemats ---
Usuari: Vision
Salt:   8ED13ABCE4A2B9B1A97700413BA11D8E
Hash:   81108C5729A6F80F813645CA4D7FF40F26D51D9160C240BC6045C182730CBD93FB6CF701AFEB0058404611A617D8B6F6AF1D79353291F
65231E0C82A5E76BB2E
```

Les dades es poden guardar inicialment en una estructura de dades en memòria.

No s'ha de guardar ni mostrar la contrasenya original una vegada finalitzat el registre.

Part 4: Inici de sessió i verificació de contrasenyes

Implementa un sistema d'autenticació que permeti als usuaris registrats iniciar sessió.

El programa haurà de:

1. **Demanar el nom d'usuari i la contrasenya.**

```
public static void iniciarSessio(Scanner sc) throws NoSuchAlgorithmException {
    // 1. Demanar usuari i contrasenya
    System.out.print("Nom d'usuari: ");
    String nom = sc.nextLine().trim();
    System.out.print("Contrasenya: ");
    String password = sc.nextLine();
```

2. **Comprovar si l'usuari està registrat.**

```
Usuari usuari = usuaris.get(nom);
if (usuari == null) {
    System.out.println("Error: usuari o contrasenya incorrectes.");
    return;
}
```

3. **Recuperar la salt i el hash associats a l'usuari.**

```
Usuari usuari = usuaris.get(nom);
```

4. **Calcular el hash de la contrasenya introduïda utilitzant la salt emmagatzemada.**

```
String hashCalculat = calcularHash(usuari.salt(), password);
```

5. **Comparar el hash calculat amb el hash guardat.**

```
boolean correcte = MessageDigest.isEqual(
        hashCalculat.getBytes(StandardCharsets.UTF_8),
        usuari.hash().getBytes(StandardCharsets.UTF_8));
```

6. **Permetre l'accés únicament quan els hashes coincideixin.**

```
if (correcte) {
    System.out.println("Accés concedit. Benvingut/da, " + usuari.nom() + "!");
} else {
    System.out.println("Error: usuari o contrasenya incorrectes.");
}
```

Si les credencials no són correctes, el programa haurà de mostrar un missatge d'error adequat.

Part 5: Proves de la pràctica

Realitza les proves següents i documenta'n els resultats:

- **Registra dos usuaris diferents amb la mateixa contrasenya. Comprova si obtenen hashes diferents.**

```
Nom d'usuari: Carlos
Contrasenya: uno
Usuari registrat.
Salt: 579B8A8A04347264ACBDCE64E69F0706
Hash: 3D17AD90F700C7251FDE3172F69C22BD2870A9FD94FC8A515C8A6CCDC283A4C72B5B7F32EBD603CEB98A988A72C15BDAA9D47C257F315496545D032289B1A975
```

```
Nom d'usuari: Pedro
Contrasenya: uno
Usuari registrat.
Salt: 6E140B4E37529E229962068BB159577D
Hash: F5F68556BB3FEE9E44D25D1A7EAF4F431243932CB2EE57B6B0C197032F49D7DF70D592EB0A67ACDE7CB7CD87C27EF60B25775BF9E1CFC8D9A937B2790D18F9B7
```

- **Intenta iniciar sessió amb una contrasenya correcta.**

```
Nom d'usuari: Pedro
Contrasenya: dos
Usuari registrat.
Nom d'usuari: Pedro
Contrasenya: dos
Accés concedit. Benvingut/da, Pedro!
Adéu!
```

- **Intenta iniciar sessió amb una contrasenya incorrecta.**

```
Nom d'usuari: Carlos uno
Contrasenya: uno
Error: usuari o contrasenya incorrectes.
Adéu!
```

- **Comprova què passa quan s'utilitza una salt diferent de la que s'ha guardat.**

```
Nom d'usuari: Pedro
Contrasenya correcta de l'usuari: dos

Hash guardat:            F5F68556BB3FEE9E44D25D1A7EAF4F431243932CB2EE57B6B0C197032F49D7DF70D592EB0A67ACDE7CB7CD87C27E
F60B25775BF9E1CFC8D9A937B2790D18F9B7

Salt guardada:           6E140B4E37529E229962068BB159577D
Hash amb salt guardada:  590B237DA7F9F4EEF162F4902C252731AD2E90544299ADEA1EC4DB94A23FDB76E7692E03F4DA0DCFB1E189685D86
A88D64A29BE9C43413B4CC4DB5D365F98E82
Coincideix? false

Salt diferent:           D6F4FA45740E73146A4641838D2A9030
Hash amb salt diferent:  BE7FA01DFADC7E79DABCC9F35DA801EF13776188256212FA6FA65B751E11133EECE522D751743CC071B900A2137A
DA8455EF5144E0C970B184A0428B74354668
Coincideix? false
```

- **Explica per què les salts han de ser diferents per als diferents registres i per què cal conservar la salt de cada usuari.**

  Si dos usuaris tenen la mateixa contrasenya i el mateix salt, el seu hash és idèntic. Un atacant que robi la base de dades veuria directament qui comparteix contrasenya, i en obtenir una les tindria totes.